

Singular quadrature in `QuadElemList` the self- and near-interaction algorithms, step by step

1 Setting

A `QuadElemList` holds high-order quadrilateral surface patches. Each patch carries an $N \times N$ tensor grid of Gauss–Legendre nodes on the reference square $[0, 1]^2$, and the surface is the order- N polynomial interpolant of those nodes,

$$\mathbf{X}(u, v) = \sum_{i,j} \mathbf{x}_{ij} L_i(u) L_j(v), \quad (u, v) \in [0, 1]^2.$$

A boundary-integral operator needs, for a target $\mathbf{y}$ and a patch E ,

$$U(\mathbf{y}) = \int_E K(\mathbf{y}, \mathbf{x}) \sigma(\mathbf{x}) dS(\mathbf{x}),$$

assembled as a matrix acting on the patch’s nodal density values. When $\mathbf{y}$ is far from E a smooth tensor rule suffices. Two cases need special treatment, and each has its own algorithm:

	self	near
target	<i>on</i> E , at a node (u_0, v_0)	<i>off</i> E , at distance $d \ll$ patch size
integrand	singular, $K \sim 1/r$	nearly singular, sharply peaked
scheme	Duffy edge-collapse	split-at-foot refinement
entry point	<code>SelfInterac</code>	<code>NearInterac</code>

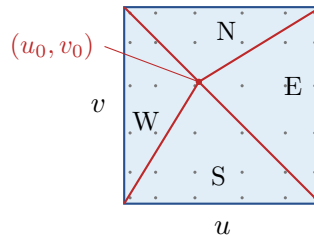
Both entry points map the runtime element order to a compile-time `order` and the runtime tolerance to a runtime `digits`; both are independent and may be called in either order.

2 Self interaction: the Duffy edge-collapse

The target sits *on* the patch, at node (u_0, v_0) , so $r = |\mathbf{X}(u, v) - \mathbf{X}(u_0, v_0)|$ vanishes there and $K \sim 1/r$ is not integrable by any smooth rule. The Duffy transform removes the singularity through the Jacobian rather than by refinement.

Step 1 — split the panel at the target

Join the target to the four corners. This gives four triangles, each with the target as one vertex and one edge of the square opposite it.



Step 2 — collapse one edge onto the target

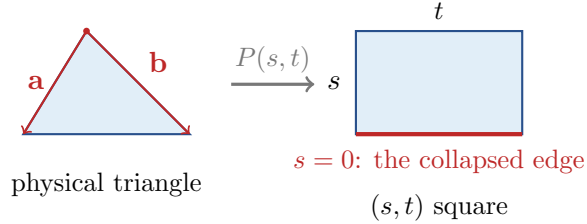
Parametrise each triangle by $(s, t) \in [0, 1]^2$ with

$$P(s, t) = (u_0, v_0) + s \mathbf{c}(t), \quad \mathbf{c}(t) = (1 - t) \mathbf{a} + t \mathbf{b},$$

where $\mathbf{a}$ and $\mathbf{b}$ point from the target to the two ends of the opposite edge. The line $s = 0$ collapses the entire t -edge onto the target. The Jacobian is

$$|\det| = s |\mathbf{a} \times \mathbf{b}|$$

— linear in s , and *independent of t* .



That single factor of s is the whole point. For a kernel of order $1/r$, writing $\|\cdot\|_G$ for the length induced by the surface metric G at the target,

$$K |\det| \sim \frac{1}{s \|\mathbf{c}(t)\|_G} \cdot s |\mathbf{a} \times \mathbf{b}| = \frac{|\mathbf{a} \times \mathbf{b}|}{\|\mathbf{c}(t)\|_G},$$

which is bounded as $s \rightarrow 0$. The singularity is gone.

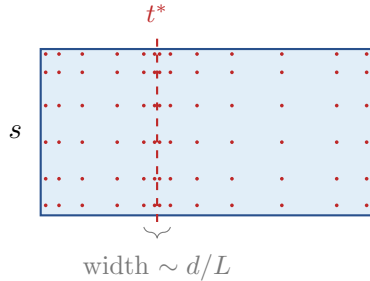
Step 3 — pick the two rules

The s -direction is now smooth, so it takes a plain Gauss–Legendre rule with $q_s = N$ nodes. The t -direction keeps a residual peak: $1/\|\mathbf{c}(t)\|_G$ is largest where $\mathbf{c}(t)$ is shortest, at the foot t^* of the perpendicular from the target to the far edge. The peak has height $\sim 1/d$ and width $\sim d/L$, where d is the distance to that edge and L its length.

Both t^* and d/L are computed in the *surface* metric G , not in parameter space: using parameter distances misplaces the peak by $|\cot \theta|$ peak-widths on a sheared patch. The peak is then resolved by a sinh substitution,

$$t = t^* + \frac{d}{L} \sinh \xi,$$

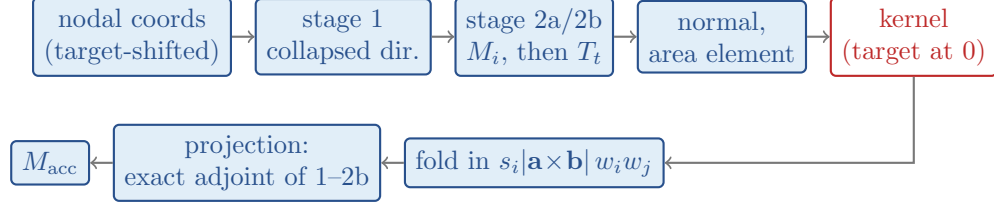
with one GL rule of n_t points in ξ . n_t grows with the requested digits and is larger for vector kernels than scalar ones.



quadrature points in (s, t) : uniform in s , sinh-clustered at t^*

Step 4 — contract, evaluate, project

Everything above depends only on $(N, t_i, t_j, \text{triangle})$, so the interpolation operators are built once per order and cached. The collapse gives them structure: for the S triangle $v(s, t) = v_0(1 - s)$ depends on s alone, so that factor is a small $(N \times n_s)$ operator rather than $(N \times n_s n_t)$. The evaluation is then a chain of GEMMs:



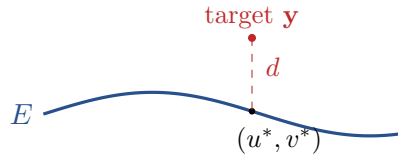
The projection reuses the same operators in reverse order, so it is the exact adjoint of the interpolation — which is what keeps the assembled operator consistent. Sources are stored relative to the target, so the kernel is always evaluated with the target at the origin and r stays accurate near the singularity.

3 Near interaction: split at the foot

Here the target is *off* the patch at a distance d that may be far smaller than the patch. The integrand is smooth but sharply peaked near the closest point, and a single tensor rule would need impractically many points. The scheme refines — but chooses *where* to refine so that every operator can be precomputed.

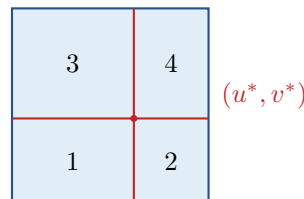
Step 1 — find the foot

Locate (u^*, v^*) minimising $|\mathbf{X}(u, v) - \mathbf{y}|^2$ over $[0, 1]^2$: seed at the nearest nodal point, then Gauss–Newton on the first fundamental form, clamped to the box with a backtracking line search, falling back to a shrinking grid search if it stalls. Optimality is tested on the *projected* gradient, since at a box edge the constrained gradient is large — it balances the constraint — and near-pair feet usually do lie on a shared patch edge.



Step 2 — split the patch *at* the foot

Cut the parameter square along $u = u^*$ and $v = v^*$, giving four sub-elements. Every sub-element now has the foot at one *corner*, so refinement always grades toward an *endpoint* of the sub-element rather than to a point in its interior.



This is the step that makes the scheme cheap. In *normalised* sub-element coordinates the graded intervals depend only on the refinement level, never on (u^*, v^*) , so their interpolation operators are built once per order and looked up. A bisection quadtree, by contrast, leaves the foot mid-cell and needs a position-dependent operator per leaf — hundreds of small matrices rebuilt for every target.

Step 3 — bisect until admissible

Within a sub-element, normalise so the foot is at $x = 1$. The graded intervals are

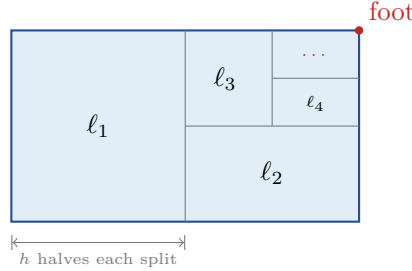
$$\text{shell}_k = [1 - 2^{-k}, 1 - 2^{-(k+1)}], \quad \text{core}_k = [1 - 2^{-k}, 1],$$

i.e. core_k is the part still touching the foot and shell_k the half turned away from it.

Splitting at the foot leaves the sub-elements *anisotropic*, so quadrisecion would hand that aspect ratio down to every descendant. Instead the corner cell is bisected along its longer *physical* dimension only — parameter extent times surface speed — one split at a time, until that dimension is admissible against the target distance:

$$b_{\text{ellipse}} \cdot \max(h_u, h_v) \leq d.$$

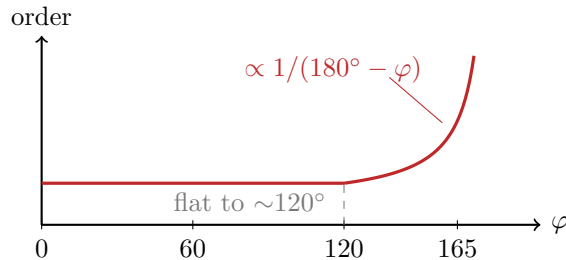
Each split emits exactly one leaf (the half not touching the corner), and the u - and v -levels advance independently.



Step 4 — choose the per-cell order

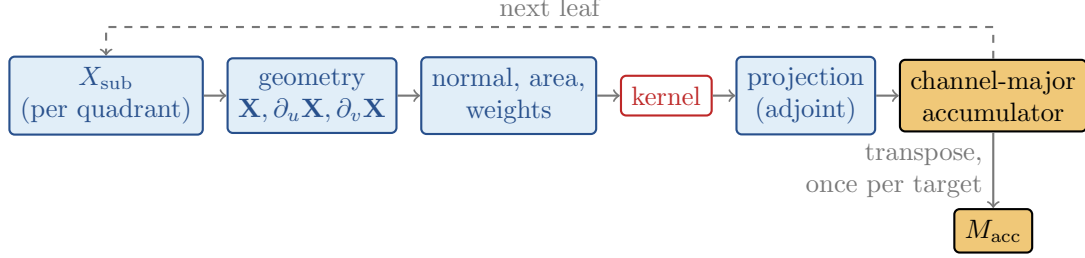
Each leaf gets a Gauss–Legendre rule whose order comes from the tolerance, via a Bernstein–ellipse argument evaluated at the *end-foot* reach (the foot lands on a cell endpoint, which is a weaker requirement than the semi-major reach by $a^2/b^2 \approx 1.9$).

That test lives in parameter space and cannot see one thing: how skewed the patch is where the target sits. The required order is flat up to a corner angle of about 120° and then grows like $1/(180^\circ - \varphi)$ as the corner flattens and the element wraps around the target. So the order is corrected per (element, target) using φ , the acute angle between the surface tangents *at the foot*. An orthogonal parametrisation gives $\varphi = 90^\circ$, a correction factor of 1, and costs a well-shaped mesh nothing.



Step 5 — integrate each leaf and accumulate

Each sub-element’s nodal coordinates are built once per target ($X_{\text{sub}} = S_u^\top \mathbf{x} S_v$), so nothing depending on (u^*, v^*) survives into the per-cell loop. Every leaf then runs the same tensor-product block:



Cells accumulate into a channel-major buffer so the projection’s last GEMM can add in place ($\beta = 1$); a single transpose into the node-major layout at the end replaces a per-cell sweep.

4 Numerical results

All from **bench-cubed-sphere** run with no arguments, which does the convergence sweep first and then the strong-scaling descent. The surface is the twisted cubed sphere of Section 3: the twist is an isometry, so the geometry is the unit sphere at every angle and only the parametrisation shears.

4.1 Convergence

Order 12, 12 patches per face (864 elements, 124,416 nodes). *error* is the on-surface Green’s identity $S[\partial_n u] - D[u] = u$, relative to $\max |u|$; *error-density* is the interpolation error of the two densities at off-node parameters, i.e. how well the order-12 patch represents them.

kernel, twist	tol	error-density	error
laplace, $\pi/6$	1.00×10^{-3}	7.18×10^{-14}	2.14×10^{-7}
	1.00×10^{-6}	7.18×10^{-14}	3.73×10^{-10}
	1.00×10^{-9}	7.18×10^{-14}	3.42×10^{-12}
	1.00×10^{-12}	7.18×10^{-14}	3.32×10^{-13}
laplace, $\pi/2$	1.00×10^{-3}	4.89×10^{-11}	2.44×10^{-7}
	1.00×10^{-6}	4.89×10^{-11}	8.52×10^{-10}
	1.00×10^{-9}	4.89×10^{-11}	4.07×10^{-11}
	1.00×10^{-12}	4.89×10^{-11}	2.71×10^{-12}
laplace, π	1.00×10^{-3}	4.28×10^{-8}	2.02×10^{-6}
	1.00×10^{-6}	4.28×10^{-8}	1.63×10^{-8}
	1.00×10^{-9}	4.28×10^{-8}	2.04×10^{-10}
	1.00×10^{-12}	4.28×10^{-8}	4.65×10^{-11}
stokes, $\pi/6$	1.00×10^{-3}	2.25×10^{-13}	4.25×10^{-6}
	1.00×10^{-6}	2.25×10^{-13}	4.66×10^{-9}
	1.00×10^{-9}	2.25×10^{-13}	1.07×10^{-10}
	1.00×10^{-12}	2.25×10^{-13}	7.68×10^{-12}
stokes, $\pi/2$	1.00×10^{-3}	1.54×10^{-10}	8.06×10^{-6}
	1.00×10^{-6}	1.54×10^{-10}	3.10×10^{-8}
	1.00×10^{-9}	1.54×10^{-10}	8.89×10^{-10}
	1.00×10^{-12}	1.54×10^{-10}	5.98×10^{-11}
stokes, π	1.00×10^{-3}	1.58×10^{-7}	1.36×10^{-5}
	1.00×10^{-6}	1.58×10^{-7}	1.10×10^{-7}
	1.00×10^{-9}	1.58×10^{-7}	8.76×10^{-10}
	1.00×10^{-12}	1.58×10^{-7}	3.19×10^{-10}

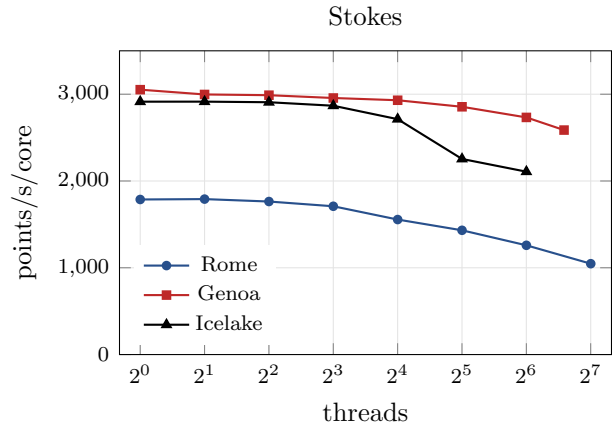
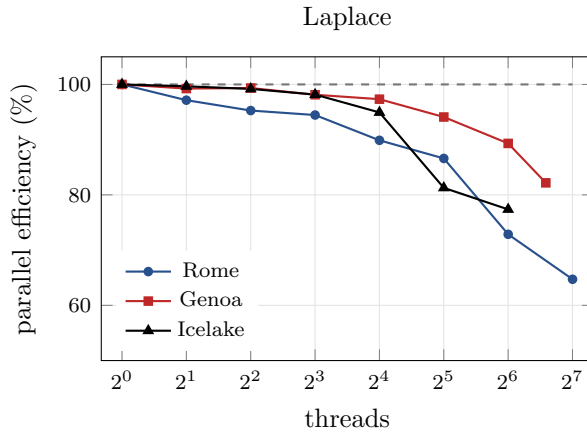
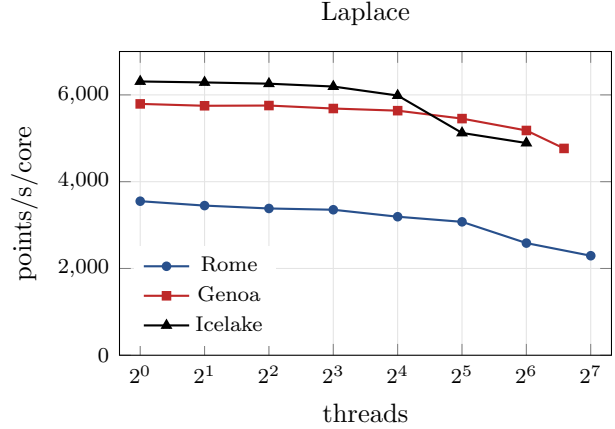
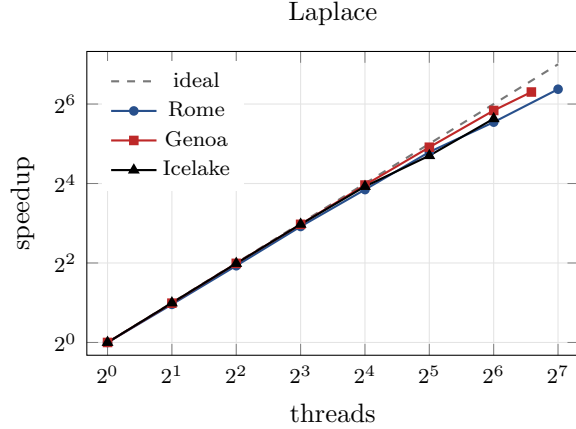
Three things to read off it. The geometry is resolved to $\sim 10^{-15}$ at every twist (not shown), so it is never the limit here. The *error* tracks the requested tolerance over nine decades — at twist $\pi/6$, $2.1 \times 10^{-7} \rightarrow 3.3 \times 10^{-13}$ — until it meets a floor set by the *parametrisation*, which degrades sharply with twist: at π the best Laplace result is 4.7×10^{-11} against 3.3×10^{-13} at $\pi/6$, a factor of 140 for the same mesh and the same tolerance. And the *error* sits well *below* the *error-density* at high twist (4.7×10^{-11} versus 4.3×10^{-8}): the identity is tested at nodes, where the densities are exact, and the quadrature averages the between-node error — which is why *error-density* is a resolution indicator and not a bound.

4.2 OpenMP scaling

Order 12, 8 patches per face (384 elements, 55,296 nodes), twist $\pi/6$, tol 10^{-9} ; **SelfInterac** plus **NearInterac** setup, timed inside the driver, with `OMP_PLACES=cores` and `OMP_PROC_BIND=close`. Three node types:

	cores	AVX-512	build
AMD EPYC 7742 (Rome)	2×64	no	<code>-march=native</code> $\rightarrow$ AVX2
AMD EPYC 9474F (Genoa)	2×48	yes	<code>-march=native</code> $\rightarrow$ AVX-512
Intel Xeon 8358 (Icelake)	2×32	yes	<code>-march=native</code> $\rightarrow$ AVX-512

Points per second per core is the cross-architecture measure: it divides out the core count, so it compares what one core achieves and how much of that survives to the full node.



machine	kernel	1 thread			full node		
		cores	setup (s)	pts/s/core	setup (s)	pts/s/core	speedup
Rome	Laplace	128	15.575	3550	0.188	2294	82.8× (65%)
Genoa	Laplace	96	9.546	5793	0.121	4766	78.9× (82%)
Icelake	Laplace	64	8.764	6310	0.177	4891	49.5× (77%)
Rome	Stokes	128	30.945	1787	0.413	1047	74.9× (59%)
Genoa	Stokes	96	18.117	3052	0.223	2587	81.2× (85%)
Icelake	Stokes	64	18.977	2914	0.410	2107	46.3× (72%)

Per core at one thread, Icelake and Genoa are 1.6–1.8× Rome (Laplace 6310 and 5793 against 3550). Rome is the only one of the three without AVX-512, and it is the only one compiled to AVX2, so that gap is the expected place for the vector width to show.

Per core at the full node the ordering is the same but the margins narrow, because all three lose throughput with width and Rome loses the most: −35% from 1 thread to 128, against −18% (Genoa, to 96) and −22% (Icelake, to 64). Genoa holds 82–85% efficiency at 96 threads where Rome is at 59–65% at 128; part of that is simply that Rome is asked to go further, but Rome is also below Genoa at matched width (73% vs 82% at 64 threads, Laplace).

In absolute terms the full node delivers 294k, 458k and 313k points/s (Laplace) on Rome, Genoa and Icelake — Genoa wins on aggregate despite having fewer cores than Rome, and Icelake matches Rome with half the cores.

Two things bound every curve and are visible in all three: at the widest setting there are only $384/N$ elements per thread in the self loop, which is coarse for `schedule(static)` (3 per thread on

Rome at 128); and `GreensDensityError` in the driver is serial, so it does not shrink with width. It is excluded from these timings, but it does mean whole-run time scales worse than the table suggests. Stokes is uniformly a little below Laplace, consistent with its larger per-target working set.

4.3 Other node types

Not yet run. `scripts/bench-cubed-sphere.sbatch` takes the architecture and the width from the submit line, so one script covers every node type:

```
Intel Icelake, 64 cores  sbatch -C icelake --cpus-per-task=64 scripts/bench-cubed-sphere.sbatch
AMD Genoa, 96 cores    sbatch -C genoa   --cpus-per-task=96 scripts/bench-cubed-sphere.sbatch
```

The scaling section runs the full width and then powers of two down to 1, so a 96-core node gives 96, 64, 32, ..., 1 and the curves stay comparable across machines with different core counts. Each architecture builds into its own `bin.<tag>/` and `obj.<tag>/` and nothing writes `./bin`. That is not tidiness: the Makefile compiles with `-march=native`, so an Icelake or Genoa binary carries AVX-512 and dies with SIGILL on the Zen 2 workstations, which share this home filesystem.

5 The two side by side

	self (Duffy)	near (split-at-foot)
kills the singularity by geometry split	the Jacobian $s \mathbf{a} \times \mathbf{b} $ 4 triangles at the target	refinement toward the foot 4 sub-elements at the foot, then bisection
cells per target	4	$\mathcal{O}(10)$, distance-dependent
what is cached	per (N, digits) : the whole triangle table	per N : the $\text{shell}_k/\text{core}_k$ operator ladder
what is per-target	the t -rule (metric-aware $t^*, d/L$)	the foot, the cell list, the corner-angle order
metric enters via	t^* and the peak width at (u_0, v_0)	surface speeds, and φ at the foot

Both share the same leaf-level machinery — interpolate to quadrature nodes, form the surface geometry, evaluate the kernel with the target at the origin, fold in weights, project back with the adjoint — and both are written so that the per-target work is GEMMs against precomputed operators rather than freshly built ones.